

```

---
title: "knn"
output: html_document
date: "2026-10-05"
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

#This occurred because it contains the knn() function that is needed to perform K-
Nearest Neighbors classification.
```{r}
library(class)

#This was used to import the data into R.
```{r}
UniversalBank <- read.csv("Downloads/UniversalBank.csv")

#This helps me make sure the data imported correctly and see what the variables look like.
```{r}
head(UniversalBank)

#This tells me how many rows and columns are in the dataset.
```{r}
dim(UniversalBank)

#This helps to identify the variables and make sure I used the correct names.
```{r}
names(UniversalBank)

#This shows the including minimums, maximums, averages, and distributions.
```{r}
summary(UniversalBank)

#Convert Education into a factor because Education represents categories.
```{r}
UniversalBank$Education <- factor(UniversalBank$Education)

#Using periods in the names makes the variables easier to reference in R.
```{r}
names(UniversalBank)[names(UniversalBank) == "ZIP Code"] <- "ZIP.Code"
names(UniversalBank)[names(UniversalBank) == "Personal Loan"] <- "Personal.Loan"
names(UniversalBank)[names(UniversalBank) == "Securities Account"] <- "Securities.Account"
names(UniversalBank)[names(UniversalBank) == "CD Account"] <- "CD.Account"

#Used to check the column names again to make sure the variables were renamed correctly.
```{r}
names(UniversalBank)

#This is used by the KNN model to predict whether a customer accepts a personal loan.
```{r}
X <- model.matrix(
  ~ Age + Experience + Income + Family + CCAvg +
    Education + Mortgage + Securities.Account +

```

```

    CD.Account + Online + CreditCard - 1,
    data = UniversalBank
)
...

#This lets me check how the variables, especially Education, were converted.
```{r}
colnames(X)
```

#This creates outcome variable Y. Personal.Loan is the variable I am trying to predict. The
0 means the customer did not accept the loan and 1 means the customer accepted the loan.
```{r}
Y <- factor(
 UniversalBank$Personal.Loan,
 levels = c(0, 1)
)
...

#I set a random seed so I get the same training and validation split each time the code is
run.
```{r}
set.seed(123)

train_index <- sample(
  1:nrow(UniversalBank),
  size = 0.60 * nrow(UniversalBank)
)

valid_index <- setdiff(
  1:nrow(UniversalBank),
  train_index
)
...

#Checked how many observations are in the training and validation datasets. This confirms
that the data was split correctly.
```{r}
length(train_index)
length(valid_index)
```

#Used to standardize the training data so the variables are on a similar scale. Standardize
the validation data using the same mean and standard deviation.
```{r}
train_means <- apply(X[train_index,], 2, mean)
train_sds <- apply(X[train_index,], 2, sd)

train_scaled <- scale(
 X[train_index,],
 center = train_means,
 scale = train_sds
)

valid_scaled <- scale(
 X[valid_index,],
 center = train_means,
 scale = train_sds
)
...

#This is information about the customer. This is information on customer's age, income,
family size, education, mortgage, accounts, online banking use, and credit card use.
```{r}

```

```

new_customer <- c(
  Age = 40,
  Experience = 10,
  Income = 84,
  Family = 2,
  CCAvg = 2,
  Education1 = 0,
  Education2 = 1,
  Education3 = 0,
  Mortgage = 0,
  Securities.Account = 0,
  CD.Account = 0,
  Online = 1,
  CreditCard = 1
)
...

#Standardize the new customer's values using the training data mean and standard deviation
so the new customer is on the same scale as the training data.
```{r}
new_customer_scaled <- (
 new_customer - train_means
) / train_sds
...

#Used to active K-Nearest neighbors.
```{r}
library(class)
...

#Run the KNN model using k = 1. This predicts the new customer's Personal Loan outcome
based on the single closest customer in the training dataset.
```{r}
prediction_k1 <- knn(
 train = train_scaled,
 test = matrix(new_customer_scaled, nrow = 1),
 cl = Y[train_index],
 k = 1
)

prediction_k1
...

#Create a sequence of possible k values from 1 through 20.I tested each value to determine
which number of neighbors performs best.
```{r}
k_values <- 1:20

accuracy <- numeric(length(k_values))

for (i in seq_along(k_values)) {

  predictions <- knn(
    train = train_scaled,
    test = valid_scaled,
    cl = Y[train_index],
    k = k_values[i]
  )

  accuracy[i] <- mean(
    predictions == Y[valid_index]
  )
}

```

```
...
```

```
#I created a data frame showing each k value and its validation accuracy.This makes it easier to compare the performance of the different k values.
```

```
```{r}
```

```
k_results <- data.frame(
 k = k_values,
 Accuracy = accuracy
)
```

```
k_results
```

```
\\`
```

```
#Found the k value that produced the highest validation accuracy.This will be used as the best k for the KNN model.
```

```
```{r}
```

```
best_k <- k_values[which.max(accuracy)]
```

```
best_k
```

```
\\`
```

```
#I used the best k value to make predictions for the validation dataset.This allowed me to analyze the final selected KNN model.
```

```
```{r}
```

```
valid_prediction <- knn(
 train = train_scaled,
 test = valid_scaled,
 cl = Y[train_index],
 k = best_k
)
```

```
...
```

```
#I created a confusion matrix comparing the actual loan outcomes with the outcomes predicted by the KNN model.In this case, it shows how many customers were classified correctly and incorrectly.
```

```
```{r}
```

```
table(  
  Actual = Y[valid_index],  
  Predicted = valid_prediction  
)
```

```
\\`
```

```
#Used the best k value to predict whether the new customer will accept the personal loan.
```

```
```{r}
```

```
prediction_best_k <- knn(
 train = train_scaled,
 test = matrix(new_customer_scaled, nrow = 1),
 cl = Y[train_index],
 k = best_k
)
```

```
prediction_best_k
```

```
\\`
```

```
#Set the seed again so the new data split can be reproduced.I stored the total number of observations in the dataset, randomly select 50% of the observations for the training dataset and I identified all observations that were not placed in the training dataset.
```

```
```{r}
```

```
set.seed(123)
```

```
n <- nrow(UniversalBank)
```

```
train2_index <- sample(  
  1:n,
```

```

    size = 0.50 * n
  )

remaining_index <- setdiff(
  1:n,
  train2_index
)

valid2_index <- sample(
  remaining_index,
  size = 0.30 * n
)

test_index <- setdiff(
  remaining_index,
  valid2_index
)
...

#Checked the number of observations in the training, validation, and test datasets.This
confirms that the data was divided into the expected proportions.
```{r}
length(train2_index)
length(valid2_index)
length(test_index)
```

#I calculated the mean and standard deviation of each predictor using the new training
dataset.
```{r}
train2_means <- apply(X[train2_index,], 2, mean)
train2_sds <- apply(X[train2_index,], 2, sd)

train2_scaled <- scale(
 X[train2_index,],
 center = train2_means,
 scale = train2_sds
)

valid2_scaled <- scale(
 X[valid2_index,],
 center = train2_means,
 scale = train2_sds
)

test_scaled <- scale(
 X[test_index,],
 center = train2_means,
 scale = train2_sds
)
...

#I used leave-one-out cross-validation on the training dataset.This gives me an estimate
of how well the KNN model performs on the training data.
```{r}
train_prediction <- knn.cv(
  train = train2_scaled,
  cl = Y[train2_index],
  k = best_k
)

table(
  Actual = Y[train2_index],

```

```
    Predicted = train_prediction
  )
  ...
```

#I used the training dataset to predict the outcomes in the validation dataset. The best k value selected earlier is used for the model.

```
```{r}
validation_prediction2 <- knn(
 train = train2_scaled,
 test = valid2_scaled,
 cl = Y[train2_index],
 k = best_k
)

table(
 Actual = Y[valid2_index],
 Predicted = validation_prediction2
)
...

```

#I used the final KNN model to predict the Personal Loan outcomes for the test dataset.

```
```{r}
test_prediction <- knn(
  train = train2_scaled,
  test = test_scaled,
  cl = Y[train2_index],
  k = best_k
)

table(
  Actual = Y[test_index],
  Predicted = test_prediction
)
...

```

#I plot the validation accuracy for each value of k. This allows me to visually compare the performance of different numbers of neighbors. The highest point on the graph represents the k value with the best validation accuracy.

```
```{r}
plot(
 k_results$k,
 k_results$Accuracy,
 type = "b",
 xlab = "Number of Neighbors (k)",
 ylab = "Validation Accuracy",
 main = "Validation Accuracy for Different k Values"
)
...

```